

Наследяване - механизъм, чрез който един клас (наричен *derived*) може да придобие свойства и поведение на друг клас (*base*)

- Концепцията (връзката) е *is-a* (Студент е човек)

Полиморфизъм - различни класове могат да бъдат третирани като базов клас, ако имат виртуални функции

Видове наследяване:

- *public* - публичните членове на базата остават публични
- *protected* - публичното на базата става защитено
- *private* - публичното на базата става частно
- *multiple* (множествено наследяване) - наследяване от няколко базисни класа
- *virtual* (виртуално) - използва се при диамантен проблем, за да се избегне дублиране на обща база.

Object slicing - възниква, когато обект от наследен клас се копира или подава по стойност като обект от базов клас.

⇒ губи се частта на наследения клас (данните и поведението му)

- Случва се при: подаване на параметри по стойност (*Base b*)
връщане на обект по стойност като базов тип
съхранение в контейнер от базов тип по стойност (*vector<Base>*)

Конструктори при наследяване - извикват се от базовия към наследения клас

1. Конструкторите на базовите класове
2. Конструкторите на член-данните (в реда, в който са декларирани)
3. Конструктора на самия произведен (наследен) клас

Копиране при наследяване - първо се копира базовата част след това копира членовете на производния клас

⇒ коректна инициализация на обекта

Move семантики при наследяване

- Първо *Base(Base&&)* после *Derived(Derived&&)*
- Не копира ресурсите, а ги прехвърля
- Оригиналният обект остава във валидно, неопределено състояние
- Move конструктор на *Derived = delete*, ако *Base* няма move конструктор

std::any - гарантура type safety, ownership и когато не знаем нищо за ^{типа} данни

- Контейнер, който може да държи стойност от произволен ~~и~~ тип, без да знаем типа при компилация.

std::any a → съдържа всичко, което е copyable

- Използва се за колекция от произволни типове, които не са известни по време на компилация.

- Ако типовете са известни → std::variant